

[← Go Back](#)

Hardware

03. Uploading Sketches to the M4 Core on Arduino Portenta X8
04. Data Exchange Between Python® on Linux & Arduino Sketch
05. Managing Containers with Docker on Portenta X8
06. Using FoundriesFactory® Waves Fleet Management
07. Deploy a Custom Container with Portenta X8 Manager
08. How To Build a Custom Image for Your Portenta X8
09. How To Update Your Portenta X8
10. Data Logging with MQTT, Node-RED, InfluxDB and Grafana
11. Output WebGL Content on a Screen
- 12. Multi-Protocol Gateway With Portenta X8 & Max Carrier**
13. Running WordPress & Database Containers on Portenta X8
14. Portenta X8 Firmware Release Notes
15. Edge AI Flow Monitoring on Portenta X8 with Docker
16. Getting Started with ROS 2 and Turtlesim on Portenta X8

Home / Hardware / Portenta X8 / **12. Multi-Protocol Gateway With Portenta X8 & Max Carrier**

12. Multi-Protocol Gateway With Portenta X8 & Max Carrier

This tutorial shows how to setup a multi-protocol gateway environment on Portenta X8 using Max Carrier

Author

Taddy Ho Chung and José Bagur

Last revision

02/28/2025

ON THIS PAGE

Overview

- Goals +
- Multi-Protocol Gateway Basics
- The Arduino Layer
- The Linux Layer
- Instructions +
- Conclusion +
- Troubleshooting
- Trademark Acknowledgments

Overview

Portenta X8 has the NXP® i.MX 8M Mini MPU (Linux) and STM32H747XI dual Cortex®-M7+M4 32bit low power ARM® MCU (Arduino) stacked together and can be used to design different workloads for these two different microprocessors. We will use the Portenta Max Carrier's onboard CMWX1ZZABZ-078 LPWAN module from Murata® and the Wi-Fi® connectivity from Portenta X8 to build a Multi-Protocol Gateway.

In this tutorial, we will go through the steps on how to set up both the Linux and Arduino sides. A device collecting sensor data will transfer the data via Wi-

Arduino and Linux layers, finally using LoRa® technology to send the information to *The Things Network*. We will also configure and expose a local communication lane to further expand its capability if a local sensor is desired.

Goals

Build an Arduino layer script using RPC to handle sensor readings or data traffic on Portenta X8

Build a multi-protocol script to manage MQTT protocol and LoRa® connectivity to handle data traffic on the Linux layer of Portenta X8

Assemble both layer scripts to build an operational multi-protocol gateway using Portenta X8 and Max Carrier

Required Hardware and Software

[Arduino Portenta X8](#)

[Arduino Portenta Max Carrier](#)

[USB-C® cable \(USB-C® to USB-A cable\)](#)

Wi-Fi® Access Point with Internet Access

868-915 MHz antenna with SMA connector

ADB: [Check how to connect to your Portenta X8](#)

[Multi-protocol gateway](#)

Arduino IDE 1.8.10+,
Arduino IDE 2, or Arduino
Cloud Editor

Multi-Protocol Gateway Basics

A gateway is a network node and a key point for data exchange between different networks under certain specifications. It refers to hardware with specific protocol that communicates between two networks. On the other hand, a **multi-protocol gateway** goes one step further by implementing multiple protocols in a single gateway.

The idea of **Multi-Protocol Gateway** is to build a device that will establish an information relay that handles incoming and outgoing data traffic using different connectivity protocols.

This means the gateway can receive the data transmitted in a certain protocol type and relay data in a different protocol for a remote server. Such a feature provides the ability to develop distinctive protocol types and relays the data with less complexity. An example could be an IoT gateway that takes orders from a remote server connected to the internet and transforms those orders into the *Bluetooth® Low Energy* protocol to turn on and off smart light bulbs.

The Portenta X8 paired with the

between different protocols,
being the following connectivity
tools at your disposal:

Wi-Fi® (MQTT Protocol, API
calls, custom protocols)

Bluetooth® Low Energy

LoRa®-based networks
(*The Things Network*)

NB-IoT & Cat-M1

The following image illustrates
an architecture overview of a
multi-protocol gateway
composed of Portenta X8 and
Max Carrier as the gateway
system.



Multi-Protocol
Gateway General
Architecture Overview

This tutorial focuses on
developing a multi-protocol
gateway using some connectivity
modules. Yet, this Portenta
combination still has much to
offer. To get the most out of it,
we will go step by step through
how to establish the multi-
protocol gateway and add
scalability to expand its
capability.

We will go through how to use
RPC to expose data received
from the Arduino layer to the
Linux layer if further
development requires you to
feed data to devices interfaced
to communicate with the M4

for expanding the capability of the Portenta X8 and Max Carrier. It will let you develop a gateway system where:

- 1 The Arduino layer will be the terminal to expose the received sensor data to control a local end device.
- 2 A local end-device transfers data to the Linux layer for further networking operation.

Hence the multi-protocol architecture will process and manage the data traffic with the desired protocol.

Foremost, you will learn how the multi-protocol gateway is implemented with a Portenta X8 paired with a Max Carrier. Some other tutorials will be referenced to guide you through the present tutorial, as it involves mechanisms that are extensive to cover.

The Arduino Layer

The Arduino layer extends within the M4 Core, which is the layer dedicated to real-time operations development. Thus, you can use the Arduino layer to perform Proportional-Integral-Derivative (PID) tasks and make the RPC calls to exchange data with the Linux layer. An example could be a temperature sensor driven with PID controller, which will use the actual reading as

provide an output for the control element.

To learn in-depth about how to exchange data using RPC between Arduino and Linux layer, please check out the [User](#)

 [Manual: Communication Between Linux And Arduino](#) or [Data Exchange Between Python® on Linux and an Arduino Sketch](#)

The Linux Layer

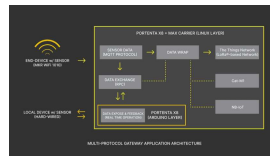
It is important to understand that **all networking processes are made within the Linux layer**. All network processes are Wi-Fi®, Bluetooth® Low Energy, LoRa® technology, NB-IoT, and Cat. M1. We will focus on using Wi-Fi® with MQTT protocol and LoRa connectivities to establish a multiple protocol gateway.

The Portenta X8 provides Wi-Fi® connectivity and the Portenta Max Carrier provides a LoRa®-based network module that can help to communicate with *The Things Network*. We will use the MQTT protocol to receive sensor data transmitted by an end device.

We will use a Python® script to configure and handle the connectivity modules and their sensor data. The RPC calls will expose the received sensor data to the Arduino layer, setting up

Portenta X8 and Max Carrier.
The process can also be done
vice-versa, using the Arduino
layer to transmit the data to the
Linux layer from the local end
device.

Now that we know the roles of Arduino and Linux layer, we will need a clear picture of how the multi-protocol gateway should look. The following diagram illustrates the in-depth multi-protocol gateway architecture, showing how each layer and module will cooperate.



Multi-Protocol Gateway In-Depth Architecture

Instructions

To showcase the ability of the Linux layer and Arduino layer extended by M4 Core, we will build a multi-protocol gateway that will receive MQTT protocol packages using EMQX as the broker, retrieve the data from a local sensor attached to Portenta via RPC mechanism, wrap the data together and send to *The Things Network* using LoRa® connectivity within Cayenne Low Power Payload Encoder.

The Cayenne Low Power Download

formatters for data exchange between end devices and has compatibility support for Arduino boards. The encoder has the capability to decode CayenneLPP payload formats without additional custom code and it helps simplify data transmission over LPWAN networks.

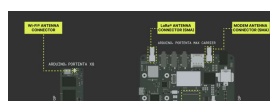
You can find more information regarding Cayenne Low Power Payload Encoder by following [here](#)



With this, following sections will help you get a better understanding of how the Portenta X8 and the Portenta Max Carrier can help you develop a multi-protocol gateway.

Hardware Setup

Before setting up the software, it is necessary to configure the hardware to be able to develop and work on the Multi-Protocol gateway. Attach the Portenta X8 to the Portenta Max Carrier High-Density Connectors and be sure to attach an antenna for LoRa® connectivity to the Portenta Max Carrier. The Portenta X8 also needs to have the Wi-Fi® antenna attached to it.



Multi-Protocol Gateway Hardware Setup

If you have not set up your Portenta X8, please have a

 look at [User Manual's Out-of-the-box experience](#)

Setting Up The Portenta X8

Before you begin diving deep into creating a Multi-protocol gateway, and having understood that you will frequently communicate between Arduino and Linux layers, you will have to understand how to debug and observe these 2 layers interact.

The `m4-proxy` is a service that manages data exchange between these layers. You can use the following command in the terminal to observe if the service is running correctly.

```
1 sudo journalctl -fu m4
```

Now you will implement RPC (Remote Procedure Call) to establish communication between the Arduino and Linux layers, which is a communication mechanism developed to exchange data between these two layers.

Serial.println() statements

to check if the Arduino sketch is running in the desired manner.

You will have to use instead

`py-serialrpc`, which is a

service that will assist you in listening to those messages, and printing them on a console on the Linux layer. To have the service active, please download [this compressed file](#) to build and run the container on the Linux side of Portenta X8. Please execute the following commands to have the service running.

```
1 // Copy the decompressed file
2 adb push py-serialrpc /home/fio/py-serialrpc
3 adb shell
4
5 sudo su -
6
7 // Head to directory
8 cd /home/fio/py-serialrpc
9 #py-serialrpc sudo docker compose up
10 #py-serialrpc sudo docker compose logs
```

To access the logs of

`py-serialrpc` service, while

maintaining the same directory, execute the following command.

```
1 sudo docker compose logs
```

For more details about how data exchange between Arduino and Linux layer works and to

 understand how to debug, please read [Data Exchange Between Python® on Linux and an Arduino Sketch](#)

If you have not configured internal Wi-Fi® connectivity within the system, please use the following command line:

```
1 nmcli device wifi conn
```

Setting Up The Things Network

You now have the prerequisites for the Portenta X8 ready, but since you are using the LoRa® connectivity, you will need a platform capable of receiving data transmitted from the Portenta X8 and Max Carrier.

The Things Network will be the platform we will use to communicate using LoRa technology. On the platform, you will need to create an application to add the Portenta Max Carrier as an End-Device.

Manual option will be used when adding the End-Device. The Portenta Max Carrier will be added under Arduino SA to be included within the LoRa®-

If you are unfamiliar with Docker and containers, please read the tutorial on how to [Create and Upload a Custom Container to the Portenta X8](#)

You can access the files [here](#).
Meanwhile, let's go through some of the important details of the included files.

Container Build File: Docker Compose

The `docker-compose.yml` file is where you define permissions and settings for the involved container. This helps to define service dependencies and provide an easy configuration. It will allow to deploy and configure for multiple containers within its defined instructions.

```
1  version: '3.6'
2
3  services:
4    multi_gateway:
5      image: multi_gate
6      container_name: '
7      ...
8      extra_hosts:
9        - 'm4-proxy:hos
10     devices:
11       - '/dev/ttymxc3
12       - '/dev/gpiochi
13     tty: true
14     user: "0"
```

In this scenario, we are granting the Portenta a major permission if it needs to access certain

several radio modules and peripherals that will connect to the sensors. It also tells to build the image file to be built locally.

Container Build File: Requirements

Here you will define which additional components are required to run the script built inside the container. If you decide to develop further with a different protocol, you will have to add the package to be able to use them for development.

```
1 msgpack-rpc-python
2 pyserial==3.4
3 python-periphery==2.3.
4 python-dotenv
5 pycayennelp
6 paho-mqtt
```

Multi-Protocol Python® Application

This is the main Python® script that will handle the overall networking process. We will highlight important lines of the code to help you understand how these code pieces work together to build a gateway based on multiple protocols. For the full Python® script, please refer to the files [here](#).

First up, is the configuration for the M4 Proxy Server, which are the parameters that handle

The `m4_proxy_port` is configured to `5001`, as it is the port used by clients to send the data to the M4.

```
1 #M4 Proxy Server Confi
2 # Fixed configuration
3 port = 8884
4 publish_interval = 5
5
6 # The M4 Proxy address
7 m4_proxy_address = 'm4
8 m4_proxy_port = 5001
```

The `get_data_from_m4()` function handles data retrieval from the Arduino layer (M4 Core). It will manage the available information to use with the Python® script. The `RpcAddress()` defines the proxy address and port for M4, which then is used as an argument for `RpcClient()` to pull the sensor data within the M4 core proxy configuration. The data is requested by indicating the name of the data variable with the `rpc_client.call(DATA)` method.

```
1 def get_data_from_m4(
2
3     rpc_address = Rpc
4
5     data = ()
6
7     try:
8         rpc_client =
9         rpc_data0 = r
10
11         rpc_client =
12         rpc_data1 = r
13
14         data = rpc_da
15
16     except RpcError.T
17         print("Unable
18
19     return data
```

For MQTT configuration, you will need to set the desired parameters. Below you can find the parameters we use for MQTT in this tutorial.

```
1 mqtt_broker = 'broker.
2 mqtt_port = 1883
3 mqtt_topic = "multiPrG
4 # Generate client ID w
5 mqtt_client_id = f'pyt
6 mqtt_username = 'emqx'
7 mqtt_password = 'publi
```

These 2 parameters are required to establish a connection with *The Things Network*. The `APP_EUI` and `APP_KEY` are required to be configured, as they are provided by *The Things Network* or from the LoRa®-based network platform that you may try to establish the

the device will request and apply the EUI. However, if it requires different `DEV_EUI`, you can make the change in this section.

```
1  # Obtained during the
2  SECRET_APP_EUI = 'XXXX'
3  SECRET_APP_KEY = 'XXXX'
```

With these parameters configured, you have secured the connection between your device and *The Things Network*. *The Things Network* would be your end-point of the data of interest. And to send data, you need to begin by gathering this data, which can be from sensors or modules with status feedback. A sensor can be attached directly communicating via the Arduino layer to receive the data, wrap it and send it to *The Things Network*. Meanwhile, you will also need to have a mechanism that will be able to intercept data sent over Wi-Fi® connectivity using an MQTT protocol.

The following tasks are the main processes that handles the MQTT protocol. It will allow to decode incoming packets from the subscribed device and buffer the data if a timeout has occurred while waiting on the MQTT packet. In this way, you will be able to receive sensor data from any external device. For example, using Arduino MKR

```

1  # MQTT protocol han
2  ...
3  def connect_mqtt() -
4      def on_connect(c
5          if rc == 0:
6              logging.
7          else:
8              logging.
9
10     client = mqtt_cl
11     client.username_
12     client.on_conne
13     client.connect(m
14
15     # Thread in the
16     client.loop_star
17     return client
18
19 def on_message(clier
20     logging.debug(f"
21
22     decoded_mqtt_pay
23     print("MQTT Payl
24
25     # `mqtt_data` is
26     mqtt_data.append
27     client.loop_stop
28

```

For LoRa® connectivity, to establish communication with *The Things Network*, we will use the Cayenne Low Power Payload Encoder as part of the process in this build.

```
1 ...
2 frame = LppFrame()
3 frame.add_temperature
4 frame.add_humidity(1,
5
6 # MQTT payload as Gen
7 frame.add_generic(2,
8
9 # Encoding packet for
10 payload = bytes(frame
11 lora_module.sendBytes
12 ...
```

Multi-Protocol Arduino (M4) Application

For the full Arduino scripts,
please refer to the files found
inside [this compressed file](#).

The following sketch is for the
Arduino layer to retrieve the
data between Arduino and Linux
layers.

```

1  /**
2   Multi-Protocol Gate
3   Name: m4_to_pythor
4   Purpose: Sketch to
5   modified according
6   and can be adapted
7   Portenta X8.
8
9   @author Arduino
10 */
11 # Arduino side sketch
12 #include <RPC.h>
13 #include <SerialRPC.h>
14
15 const long interval = 1000;
16 unsigned long previousTime = 0;
17
18 void setup(){
19   pinMode(PA_12, INPUT);
20   //RPC.begin();
21
22   Serial.begin(115200);
23   while (!Serial)
24     continue;
25   Serial.println("Setup complete");
26
27   RPC.bind("Data_0");
28   RPC.bind("Data_1");

```

The sketch above will help expose and transfer the data processed within the Linux side. Transferring data to the Linux side can be seen as a direct communication, as the sensor connected and monitored via the Arduino side will send this data over LoRa® connectivity. For this build, we will use Analog readings to emulate the functionality.

Additionally, by exposing, you will bring forth the data received within the Linux side to the Arduino side to feed the local device as a control input. It can be used to display the data if

implementation is possible within this script if you wish to develop further desired multi-protocol gateway architecture.

For MQTT publishing, we have also included a sketch that can be used with Arduino MKR WiFi 1010 to test the gateway build. Of course, the code can be extended and modified according to the requirements of the device that will collect data from a certain sensor to send over the MQTT protocol.

```
1  /**
2   Multi-Protocol Gat
3   Name: mqtt_publish
4   Purpose: Simple MC
5   but it can be char
6
7   @author Arduino
8  */
9
10 #include <ArduinoMqt
11 #include <WiFiNINA.h
12 #include "arduino_se
13
14 // Please enter your
15 char ssid[] = SECRET
16 char pass[] = SECRET
17
18 WiFiClient wifiClier
19 MqttClient mqttClier
20
21 const char broker[]
22 int port
23 const char topic[]
24
25 //Set interval for s
26 const long interval
27 unsigned long previc
28
```

It is now time to make the multi-protocol gateway run and for this, you will need to build the Docker container that will help you operate in the background on the Linux layer. Using the terminal, you can use the following commands to get the multi-protocol gateway container up and running.

You will need to have the files ready in a folder inside the `adb` directory within Arduino root.

```
1 C:\Users\#USERNAME#\Ap
```

Having the files ready in that directory, you can use the following commands to push the files to the `fio` directory inside the Portenta X8. The second command will let you navigate inside the Portenta X8.

```
1 adb push multi-protoco
2 adb shell
```

You will now build the container using the following commands. The following command will tag the container with `Multi_Protocol_Gateway_X8` as its name.

```
1 cd ../home/fio/Multi_P
```

You will be able to see the following results when the image is built successfully.



Multi-Protocol Gateway Docker Build Terminal Log

If you have created the Docker container previously and want to re-create it with new changes made outside the shell, please check that the container and its build directory are stopped and removed. It is for the convenience of having a clean working environment

After a successful container build, you will run the image. To do that, you can use the following command. This command will immediately give an output in your terminal, telling you how the Python® script is running. If you wish to have it running in the background, please add the `-d` flag at the end of the command.

```
1 sudo docker compose up
```

Finally, you will have the multi-

connectivity. Also, RPC for exchanging data between its layers. However, there are cases where you would wish to make changes by adding more functionalities, such as including Cat. M1 or NB-IoT to expand its communication spectrum. For this, you will need to stop the image. To stop the image from running, you can use the following command.

```
1 sudo docker compose do
```

Getting to know the status of the image is also crucial as it is the indicator of the state of operation. The following command brings up **active** containers and shows the status if the container restarted or stopped due to certain reasons. The second command lists built images and it will show you the components that go with the main image that you're building.

```
1 docker ps -a
```

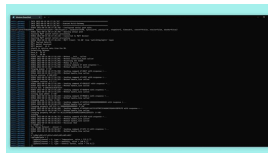
```
1 docker images
```



Multi-Protocol Gateway Active Docker Image

With all this, you have built a running multi-protocol gateway based on Portenta X8 and Max Carrier. You will be able to observe the data sent by the gateway with *The Things Network* platform End-Device section under Applications.

If you are curious about what to expect from the build you have made in this tutorial, the following image shows you the terminal of the running multi-protocol gateway.



Multi-Protocol Gateway Run Log

Conclusion

In this tutorial, you have learned how to set up a Multi-Protocol Gateway composed of MQTT protocol, RPC and LoRa® technology, by using the Portenta X8 and the Portenta Max Carrier. You have built the gateway that will connect to *The Things Network* to send the desired data. Also, the gateway is capable of exchanging data between Arduino and Linux layers using RPC, in which you

local sensor to be sent directly to *The Things Network*.

Next Steps

Now that you have developed a multi-protocol gateway, using Wi-Fi® and LoRa® connectivity, expand the gateway's capability by adding other connectivity types such as Cat. M1 and NB-IoT.

Expand functionalities for data processing using RPC while using multi-protocol architecture.

Troubleshooting

You might encounter some errors or misbehaviors while working on the code, preventing you from progressing in the development. You can try the following troubleshooting tips to solve the commonly known issues:

If the sketch upload process fails, check if your Portenta X8 is in bootloader mode. To put the Portenta X8 into Bootloader mode, double-press its RESET button and verify that the green LED is blinking. After this, you can try re-uploading the sketch for the Arduino layer.

Check the position of the BOOT DIP switch of the Portenta Max Carrier. If the Portenta X8 gets into bootloader mode immediately after powering on, including when connected via USB

OFF. This case applies to the Arduino layer.

If you encounter an issue regarding terminal input inconvenience, please

enter `export TERM=xterm`

as the command in the terminal to get readable inputs.

In case an internal Wi-Fi® connection cannot be established through the command input due to "unavailable" SSID, although it is in range. Please try using a different SSID if available or a hotspot from a different device to host network connectivity.

If you encounter a docker image conflict when running after building, please make sure you have used a name tag that matches the one from the `docker-compose.yml` file.

Trademark

Acknowledgments

LoRa® is a registered trademark of Semtech Corporation.

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository.	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

can edit
this page
[here](#).

Was this article helpful?

